

Advanced Computer Networks



Dr Sudipta Saha

Associate Professor

Dept of Computer Science & Engineering
Indian Institute of Technology Bhubaneswar

Tutorials on Congestion



**DSSRG: Decentralized
Smart Systems Research
Group**

<https://sites.google.com/iitbbs.ac.in/dssrg>

Or Google dssrg iitbbs



Self-clocking basics

A TCP connection passes through a path where the **slowest link** is a bottleneck link of rate **2 Mbps**. TCP always sends packets of size **1000 bytes**.

Assume:

- Receiver processing time is negligible and same for all packets,
- ACKs are small and preserve the same spacing
- **Sender transmits a new packet only when an ACK arrives.**

Question

1. What is the minimum packet spacing P_b on the bottleneck link?
2. What will be the ACK spacing at the sender?
3. What will be the sender's packet transmission rate in packets/sec?
4. What will be the sender's data rate in Mbps?



1. Compute bottleneck packet spacing

Packet size:

1000 bytes = 8000 bits

Bottleneck bandwidth:

2 Mbps = 2×10^6 bits/sec

So bottleneck packet time:

$$P_b = 8000 / [2 \times 10^6] = 0.004 \text{ sec} = 4 \text{ ms}$$



2. ACK spacing at sender

- By self-clocking,
- $A_s = P_b = 4 \text{ ms}$

3. Sender packet rate in packets

Packet rate = $1/0.004 = 250 \text{ packets/sec}$

4. Senders data rate

- 1 Pkt = 8000 bits, Data rate = $250 \times 8000 \text{ bits/sec} = \mathbf{2 \text{ Mbps}}$



Conclusion

- Even if the sender is connected to a much faster LAN, the **ACK clock makes it transmit at the bottleneck rate.**



Effect of bottleneck change on self-clocking

A TCP sender is initially transmitting through a path whose bottleneck link rate is **4 Mbps**. Packet size is **1500 bytes**. Later, due to a route change, the bottleneck becomes a slower link of **1.5 Mbps**.

Assume:

- Sender always sends one new packet per ACK, Receiver and reverse path preserve spacing, No losses occur.

Question

1. What is the initial bottleneck packet spacing?
2. What is the new bottleneck packet spacing after the route change?
3. What is the percentage decrease in the sender packet transmission rate?
4. If the sender transmits for 1 second after the route change, how many packets will it send?



Packet size in bits

- 1500 bytes = $1500 \times 8 = 12000$ bits

Initial bottleneck spacing

Initial bottleneck rate is 4 Mbps = 4×10^6 bits/sec

So initial spacing is

$$P_{b1} = 12000 / 4 \times 10^6 = 0.003 \text{ sec} = 3 \text{ ms}$$



New bottleneck spacing

New bottleneck rate:

$$1.5 \text{ Mbps} = 1.5 \times 10^6 \text{ bits/sec}$$

So new spacing =

$$P_{b2} = 12000 / 1.5 \times 10^6 = 0.008 \text{ sec} = 8 \text{ ms}$$

Thus the ACK spacing at sender also becomes: 8 ms

Packet rates before and after

- Initial packet rate:
- $R_1 = 1 / 0.003 \approx 333.33$ packets/sec
- New packet rate:
- $R_2 = 1 / 0.008 = 125$ packets/sec

Percentage decrease in sender packet rate

$$\begin{aligned}\text{Decrease} &= [333.33 - 125] / 333.33 \times 100 \\ &= [208.33 / 333.33] \times 100 \approx 62.5\%\end{aligned}$$



Number of packets sent in 1 second after route change

- $125 \text{ packets/sec} \times 1 \text{ sec} = 125 \text{ packets}$

Conclusion

Self-clocking automatically adapts the sender to the **new bottleneck rate**, without the sender explicitly knowing the bandwidth



Sender receiver path Asymmetry

- In real networks, the forward (data) path and reverse (ACK) path are almost never symmetric.
- Why?
- **Asymmetric Routing (Different Physical Paths)**
- Internet routing is often **asymmetric**:
- Data: $A \rightarrow B$ via path P1 while ACKs: $B \rightarrow A$ via a completely different path P2
- These paths can differ in: Number of hops, link speeds, Congestion levels
- **Effect on spacing**
- Even if forward path has spacing P_b
- **Reverse path may compress ACKs** (if faster path)
- **Stretch ACKs** (if slower path)
- So As may not be equal to P_b
- This brings some issue on self-clocking.

Let us have a numerical example on this issue



Example

- Forward path bottleneck: 2 Mbps
- Reverse path: 100 Mbps
- **Effect**
- Forward path determines packet spacing
- Reverse path can deliver ACKs **faster than they were generated**
- But since ACK generation is paced by data arrival:
- Spacing should be preserved ideally
- BUT small variations get amplified → Small jitter → **irregular ACK timing**



Delayed ACK Mechanism (Receiver Behavior)

- Modern TCP implementations (e.g., Linux TCP stack) use **delayed ACKs**:
- Instead of ACK per packet → Receiver sends **1 ACK for 2 packets**
- **Effect** → If packets arrive every 4 ms:
- ACKs may come every:
- 8 ms
- Sender now sends: Bursts of 2 packets every 8 ms
- So:
- $A_s = 2 \times P_b$



ACK Loss or Reordering

- ACKs can be: dropped or reordered
- **Effect**
- Sender behavior:
- Missing ACK → Pauses or retransmits
- Reordered ACK → Irregular triggering
- This may also break smooth self-clocking



Cross Traffic on Reverse Path

Even if forward path is clean, reverse path may carry:

- video streams
- Other TCP flows
- Control traffic

Effect

- ACKs get:
- Queued
- Delayed unevenly
- Spacing becomes: irregular
- Burst transmission at sender

TCP learns the correct *speed* from the forward path, but the *rhythm* of sending is dictated by the reverse path.



Self-Clocking under Asymmetric Paths

Consider a TCP connection between a sender and receiver where the **forward path and reverse path have the following characteristics:**

Forward

Bottleneck link bandwidth = **2 Mbps**

Packet size = **1000 bytes**

Packet spacing is determined by the bottleneck

Reverse path (ACKs):

Has a lower bandwidth link of **0.5 Mbps**

ACK size = **100 bytes**

Due to congestion on the reverse path, ACKs get queued and are released in **bursts of 3 ACKs every 12 ms**

- Sender transmits one packet per ACK received
- No packet loss
- No delayed ACK at receiver



Questions

- (a) What is the **ideal packet spacing** on the forward path (in ms)?
- (b) Under ideal self-clocking (no reverse path distortion), what should be the **ACK spacing at the sender**?
- (c) Given the actual reverse path behavior (bursts of 3 ACKs every 12 ms), describe the **sender's transmission pattern**.
- (d) Compute the **average sending rate (packets/sec)** at the sender under this bursty ACK arrival.
- (e) Compare this rate with the **ideal rate** from part (b). Is the throughput affected?
- (f) Explain briefly whether self-clocking is **preserved, partially preserved, or broken**, and why.



a) Forward path packet spacing

Packet size: 1000 bytes = 8000 bits

Bandwidth: 2 Mbps = 2×10^6 bits/sec

$$P_b = 8000 / [2 \times 10^6] = 0.004 \text{ sec} = 4 \text{ ms}$$

Answer: 4 ms



(b) Ideal ACK spacing at sender

- In ideal self-clocking:
- $A_s = P_b = 4 \text{ ms}$

Answer: 4 ms



(c) Actual sender transmission pattern

Given: 3 ACKs arrive together every 12 ms

Sender behavior:

Sends **3 packets back-to-back instantly**

Then **waits 12 ms**

Repeats

So pattern is:

Burst of 3 packets → Idle → Burst → ...

(d) Average sending rate

In each 12 ms interval:

3 packets sent

So, rate = $3 / 0.012 = 250$ packets/sec

(e) Comparison with ideal rate

Ideal rate:

$1 / 0.004 = 250$ packets / sec

So: Ideal = 250 packets/sec Actual = 250 packets/sec

Answer:

Throughput is **unchanged**



(f) Self-clocking status

Rate is preserved → bottleneck still controls throughput

But timing is distorted → ACK bursts → packet bursts

So: Self-clocking is **partially preserved**

Rate information preserved

Smooth spacing is lost

Answer: Self-clocking is **partially preserved** (rate correct, timing distorted due to reverse-path asymmetry)



Conclusion

- Reverse path asymmetry does not change the **average sending rate**, but it **destroys smooth packet pacing**, leading to irregularity and potential congestion



Further reasoning

Forward path (data packets)

Rate = **2 Mbps**

Packet size = 1000 bytes = 8000 bits

Packet spacing:

$$P_b = 8000 / 2 \times 10^6 = 4 \text{ ms}$$

So ACKs are generated every **4 ms**.

Reverse path (ACKs)

ACK size = 100 bytes = 800 bits

Bandwidth = **0.5 Mbps = 5×10^5 bps**

Service time per ACK: $800 / 5 \times 10^5 = 0.0016 \text{ sec} = 1.6 \text{ ms}$

Compare arrival vs service

ACK arrival spacing = **4 ms**

ACK service time = **1.6 ms**

Arrival rate < service rate

Meaning:

Link is **under-utilized**

No queue builds

No extra delay accumulates



Further reasoning

- Even though reverse bandwidth (0.5 Mbps) is smaller than forward (2 Mbps):
- ACKs are much smaller → require far less bandwidth
- So: *The reverse path is still fast enough to handle ACK traffic smoothly.*



When would ACKs get delayed?

ACK delay (and spacing distortion) happens if:

Case 1: Reverse path overloaded

Condition:

ACK arrival rate $>$ reverse link capacity

Example:

If ACK size = 500 bytes or reverse bandwidth = 0.1 Mbps

Then:

service time $>$ inter-arrival time

It will cause - queue build ups and ACK spacing will change

A slower reverse link does NOT distort self-clocking unless it becomes a bottleneck for ACKs



Q: Slow Start Evolution (Basic CWND Growth)

A TCP connection starts with:

Initial congestion window $CWND = 1 \text{ MSS}$, $RTT = 100 \text{ ms}$

Slow start threshold $SSThresh = 32 \text{ MSS}$

No packet loss occurs.

Question:

- a) Calculate the value of $CWND$ after each RTT until it reaches $SSThresh$
- b) How much total data (in MSS units) is transmitted by the end of the slow start phase?

(Hint: **$CWND$ doubles every RTT in slow start**)



Solution:

Given:

Initial CWND = 1 MSS, RTT=100 ms

SSThresh = 32 MSS, No packet loss

In slow start, CWND doubles every RTT

RTT number	CWND (MSS)
------------	------------

Start	1
-------	---

After 1 RTT	2
-------------	---

After 2 RTTs	4
--------------	---

After 3 RTTs	8
--------------	---

After 4 RTTs	16
--------------	----

After 5 RTTs	32
--------------	----

TCP reaches SSThresh = 32 MSS after **5 RTTs**.

Since each RTT is 100 ms, total time to reach

$$5 \times 100 \text{ ms} = 500 \text{ ms}$$



2) Total data transmitted by the end of slow start

In each RTT, sender can send a full window of data equal to the current CWND.

So total transmitted data:

$$1 + 2 + 4 + 8 + 16 + 32 = 63 \text{ MSS}$$

Time to reach threshold: **500 ms**

Total data transmitted: **63 MSS**



Q: Bottleneck Buffer Overflow Scenario

Consider an experimental set up with 4 TCP connections

Each connection window = 32 packets

Bottleneck buffer capacity = 50 packets

- **Question:**

1. Calculate the total number of packets injected into the network.
2. Determine how many packets will overflow the buffer.
3. Compute the percentage overflow relative to buffer size.



Solution:

4 TCP connections

Each window = 32 packets

Bottleneck buffer capacity = 50 packets

This matches the example where $32 \times 4 = 128$ packets may be injected toward a 50-packet bottleneck queue.

(1) Total packets injected

$$4 \times 32 = 128 \text{ packets}$$

(2) Number of packets that overflow

If the bottleneck can queue only 50 packets, then overflow is:
 $128 - 50 = 78$ packets

3) Percentage overflow relative to buffer size

$$78/50 \times 100 = 156\%$$



TCP AIMD Behavior After Packet Loss

- A TCP connection behaves as follows:
- At time t , $CWND = 40 \text{ MSS}$
- Packet loss occurs
- $SSThresh = CWND/2$
- $CWND$ resets to 1 MSS
- **Question:**
 1. Compute the new $SSThresh$.
 2. Show the $CWND$ evolution (per RTT) until it reaches $SSThresh$ again.
 3. After reaching $SSThresh$, how many RTTs are needed to increase $CWND$ from $SSThresh$ to 50 MSS (in congestion avoidance phase)?



Solution

- **Given:**
- Before loss, $CWND = 40$, Packet loss occurs
- $SSThresh = CWND/2$
- TCP resets $CWND=1$
- $CWND = 1 \text{ MSS}$ after loss
- **New $SSThresh$** $SSThresh = 40/2 = 20 \text{ MSS}$



CWND evolution until it reaches SSThresh again

- After loss, TCP restarts from $CWND = 1 \text{ MSS}$ and doubles every RTT during slow start.

RTT number after restart	CWND (MSS)
Start	1
After 1 RTT	2
After 2 RTTs	4
After 3 RTTs	8
After 4 RTTs	16
After 5 RTTs	32



- Now, strictly following the doubling rule, TCP jumps from 16 to 32, which crosses 20.
- **It reaches or exceeds SSThresh after 5 RTTs**

So:

- $SSThresh = 20 \text{ MSS}$
- Slow start lasts until the window reaches/exceeds 20 MSS this takes **5 RTTs**



RTTs needed to increase from SSThresh to 50 MSS in congestion avoidance

- In congestion avoidance, CWND increases linearly by **1 MSS per RTT**.
- Starting from 32 MSS it reaches 50 through linear increase $50 - 32 = 18$ RTT.
- TCP does **NOT** stop exactly at 20
- It **overshoots** and reaches **32**
- “Slow start until CWND = SSThresh”
- But CWND often **overshoots SSThresh**
- TCP tries to reach safe region **quickly (exponential)**
- Then switches to **careful linear growth**



Increase/Decrease function

Two users share a bottleneck link.

Let their allocations at time t be $(x_1(t), x_2(t))$

The network uses a binary feedback:

$y(t) = 0$ if $x_1(t) + x_2(t) \leq K$ (underloaded)

$y(t) = 1$ if $x_1(t) + x_2(t) > K$ (overloaded)

The knee is at:

$K=100$

So the **efficiency line** is:

$$x_1 + x_2 = 100$$

The **fairness line** is:

$$x_1 = x_2$$



Initial state: $(x_1(0), x_2(0)) = (10, 70)$

Consider the following two candidate control laws.

Scheme A

- If $y(t) = 0$, $x_1(t+1) = x_1(t) + 8$, $x_2(t+1) = x_2(t) + 8$
- If $y(t)=1$, $x_1(t+1) = 0.5 x_1(t)$, $x_2(t+1) = 0.5 x_2(t)$

• Scheme B

- If $y(t) = 0$, $x_1(t+1) = x_1(t) + 8$, $x_2(t+1) = x_2(t) + 8$
- If $y(t) = 1$, $x_1(t+1) = x_1(t) - 8$, $x_2(t+1) = x_2(t) - 8$

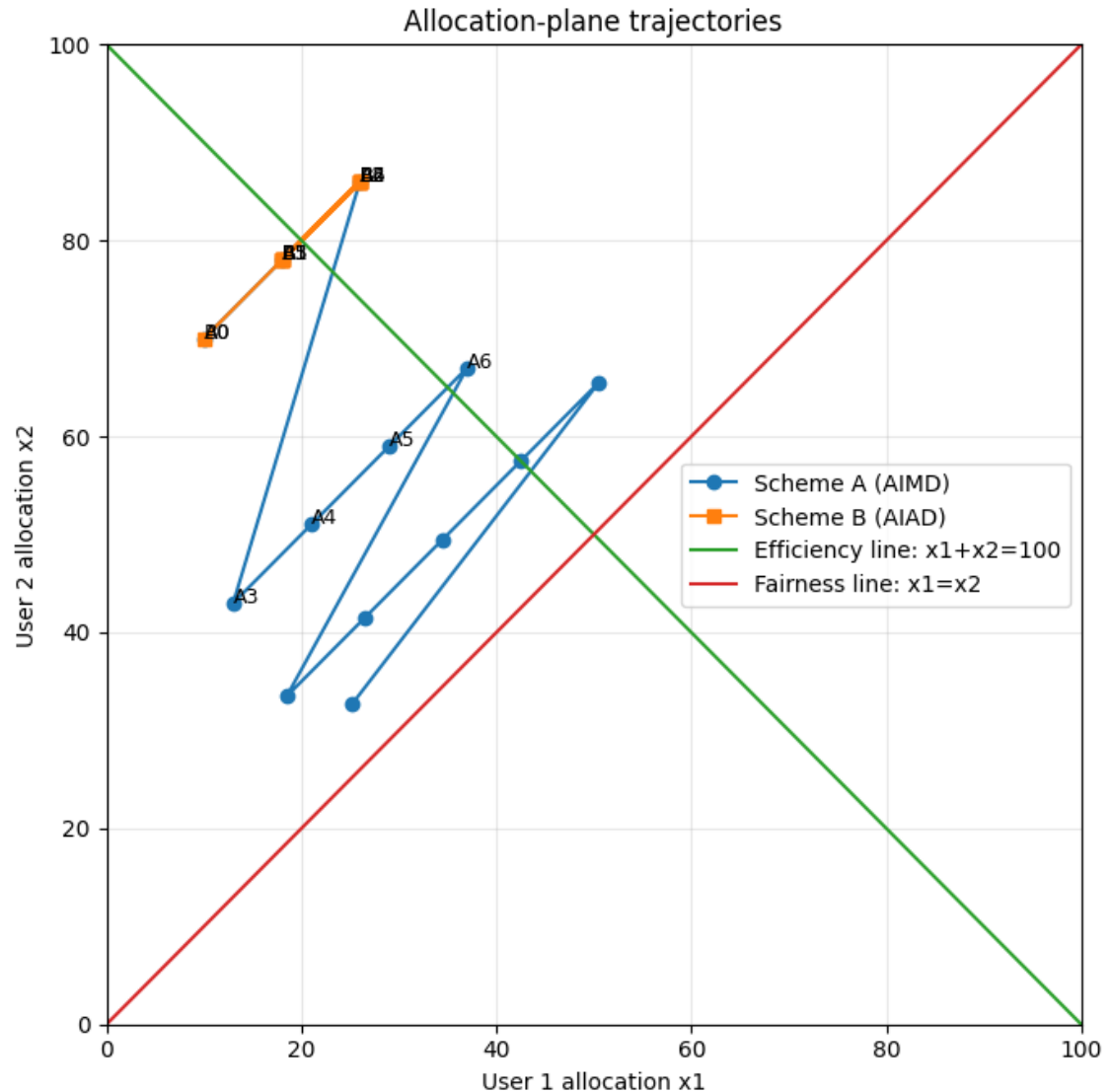
Questions

1. For each scheme, compute the first few states using the feedback rule.
2. Determine whether the scheme converges toward the fair-efficient operating point (50,50)
3. Compare the **speed of convergence** using:
 - a. Closeness to the efficiency line $x_1 + x_2 = 100$
 - b. Increase in fairness $F(t)$
4. Interpret the behavior graphically.



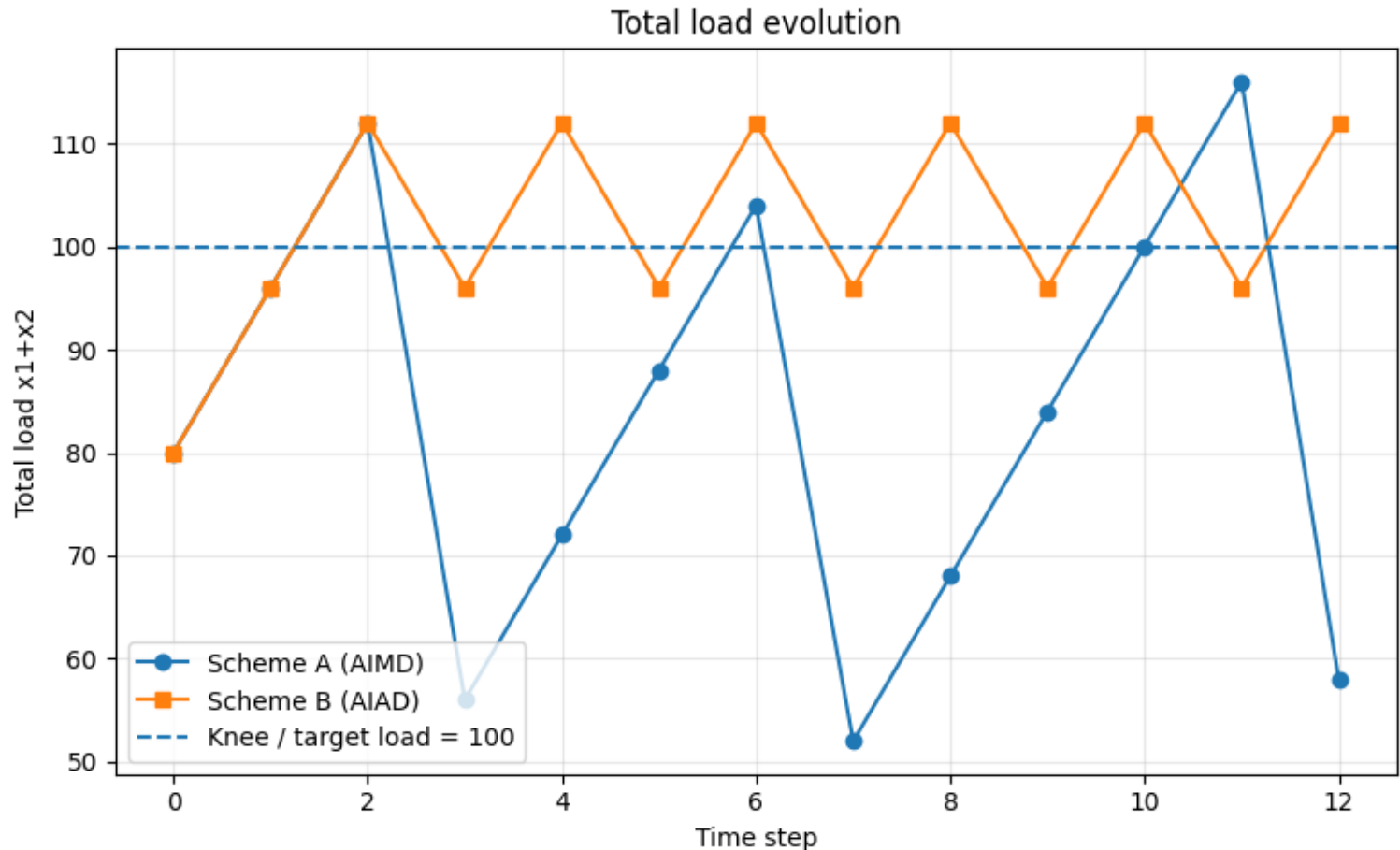
1) Allocation-plane trajectory

- How each scheme moves in the (x_1, x_2) plane relative to the **efficiency line** and **fairness line**.
- Allocation-plane trajectories



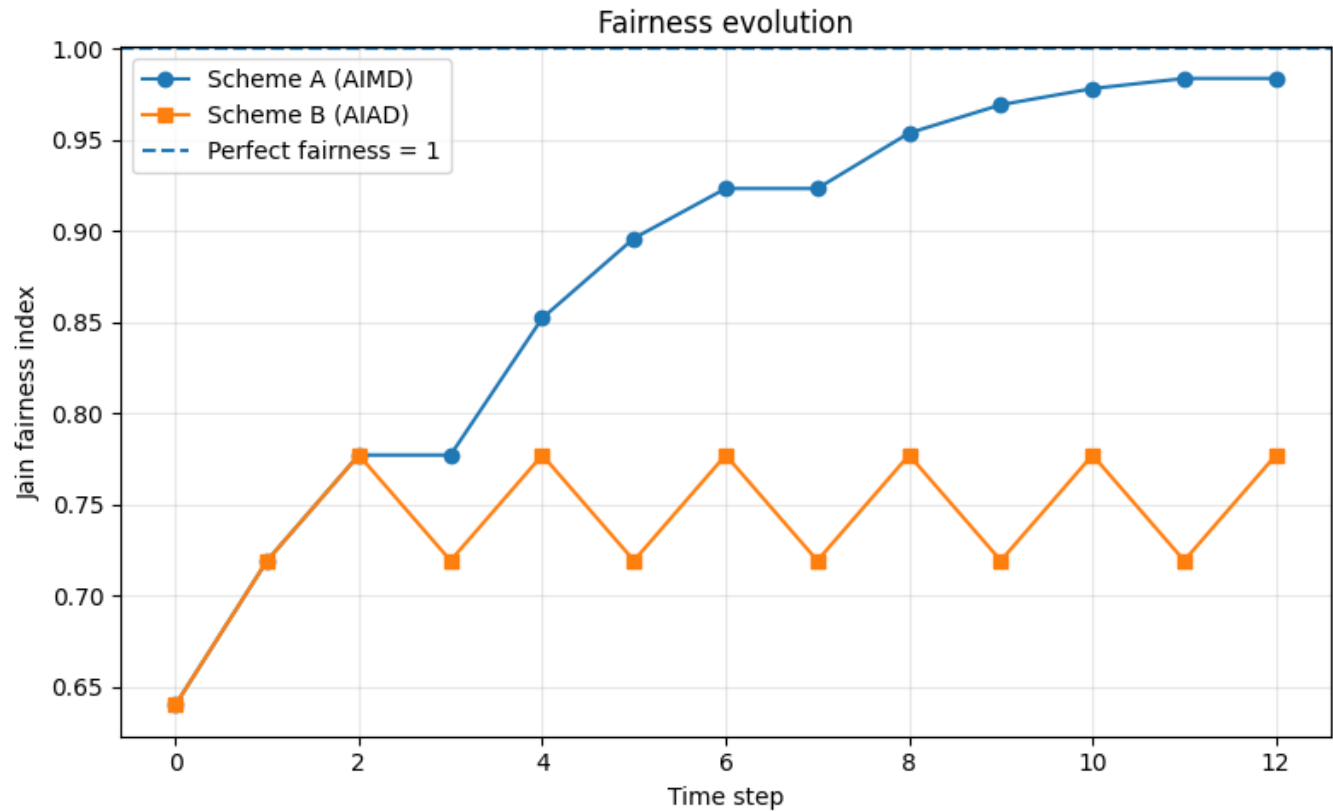
2) Total load evolution

- How quickly each scheme approaches or oscillates around the **knee** $K = 100$



3) Fairness vs time

- Whether the scheme becomes more fair over time.



Step-by-step evolution

- **Scheme A**
- Start:
- $(10,70)$, $x_1+x_2 = 80 < 100 \rightarrow y = 0$
- So additive increase applies:
- $(10,70) \rightarrow (18,78) \rightarrow (26,86)$
- Now: $26 + 86 = 112 > 100 \rightarrow y=1$
- So multiplicative decrease applies:
- $(26,86) \rightarrow (13,43)$
- Then additive increase resumes:
- $(13,43) \rightarrow (21,51) \rightarrow (29,59) \rightarrow (37,67) \rightarrow (45,75)$



- Again overload occurs, then multiplicative decrease pulls the point inward along an equi-fairness line.
- **Summary:**
 - Additive increase moves diagonally upward, improving fairness and increasing total load;
 - Multiplicative decrease moves toward the origin while preserving fairness.
 - This is the classic AIMD mechanism.



Scheme B

- Start: $(10,70) \rightarrow (18,78) \rightarrow (26,86)$
- Since $26 + 86 = 112 > 100$ overload feedback causes additive decrease:
- $(26,86) \rightarrow (18,78)$
- Then: $(18,78) \rightarrow (26,86) \rightarrow (18,78) \rightarrow \dots$
- So Scheme B gets trapped in a **two-state oscillation**.



Convergence result

Scheme A

It **does converge toward fairer operation**

It does **not settle to one fixed point**, because the feedback is binary, Instead, it oscillates around the desired operating region, with fairness improving cycle by cycle

It becomes both near the knee and reasonably fair around **time step 10**

Final fairness in the plotted run is about:

$$F \approx 0.984$$

This is very close to perfect fairness



Convergence result

- **Scheme B**
- **It does not converge to the fair-efficient point**
- It oscillates between two nearby load states
- Its fairness stays much lower:
- $F \approx 0.72$ to 0.78
- So Scheme B reaches efficiency neighborhood sometimes, but **not fairness**.



How quickly do they converge?

There are two notions here.

In terms of efficiency

Scheme B reaches the neighborhood of the knee very quickly and keeps bouncing around it.

Scheme A may overshoot and undershoot more widely at first.

In terms of fairness + efficiency together Scheme A is better

Scheme B is faster only in a narrow total-load sense, but it fails to move toward the fairness line.

So the proper conclusion is:

Scheme A converges more slowly in load, but correctly in fairness + efficiency

Scheme B responds quickly in load, but does not converge to the desired fair-efficient operating point



Conclusion

Scheme A (AIMD): zig-zag trajectory, repeated crossing of the efficiency line, gradual drift toward the fairness line, Convergence to a fair oscillatory equilibrium

Scheme B (AIAD): Fast back-and-forth motion near the efficiency boundary, no meaningful movement toward the fairness line, no convergence to the optimal point (50,50)

So, **Scheme A converges**, while **Scheme B does not**, and Scheme A is the right congestion-control style under binary feedback.

